

Machine-Coding Solutions

Six fully coded LLD problems — runnable Python with clean OOP, design patterns and extensibility notes.



LRU Cache

Parking Lot

Rate Limiter

Splitwise

Elevator

Tic-Tac-Toe

Contents

01 The Machine-Coding Approach

02 1. LRU Cache (+ LFU Variant Note)

03 2. Parking Lot

04 3. Rate Limiter

05 4. Splitwise / Expense Sharing

06 5. Elevator System

07 6. Tic-Tac-Toe (Clean OO Board Game)

08 Common Patterns Across LLD Problems

The Machine-Coding Approach

Time-box strategy (45–90 min round)

Phase	Time	Goal
Clarify requirements	5 min	Pin scope; avoid building the wrong thing
Extract entities & relationships	5 min	Noun-verb extraction on whiteboard
Sketch class diagram	5–10 min	Get agreement before coding
Code core classes + happy path	20–30 min	Must compile/run
Add edge cases + extensibility	10 min	Show you think beyond happy path
Demo driver	5 min	Prove it works end-to-end

What interviewers grade (roughly in priority order)

1. **Working, runnable code** — broken code is an automatic downgrade.
2. **Clean abstractions** — classes with single responsibilities; meaningful names.
3. **Extensibility** — adding a new feature should require adding code, not changing existing code (OCP).
4. **Edge cases handled** — not just happy path (full parking lot, divide-by-zero in splits, empty elevator queue).
5. **Design patterns named** — shows vocabulary; don't force patterns but do name what you use.

What to deprioritize under time pressure

- Persistence / DB layer (mention it, skip it).
- Concurrency / thread-safety (mention locks, skip implementation unless asked).
- Perfect error messages (simple exceptions are fine).
- Over-engineering: 3-layer factory + abstract-factory combos for a 45-min round are a red flag.

The noun-verb extraction recipe

From the problem statement: circle **nouns** → candidate classes; underline **verbs** → candidate methods. Then prune: nouns that are just attributes (e.g. "name", "price") become fields, not classes.

1. LRU Cache (+ LFU Variant Note)

Requirements

Functional - `get(key)` — return value or -1 if not present. - `put(key, value)` — insert; if capacity exceeded, evict **least recently used** entry. - Both operations must be **O(1)**.

Clarifying questions to ask - Is capacity fixed at construction? (Yes.) - Thread-safe? (Assume single-threaded unless asked; mention `threading.Lock` for MT.) - Key/value types? (Generics / Any for flexibility.) - On `put` of existing key, update value and recency? (Yes.)

Core entities / classes

Class	Responsibility
DLinkedList	Doubly-linked list node (key, value, prev, next pointers)
LRUCache	HashMap + doubly-linked list; exposes <code>get/put</code>

Class diagram (ASCII UML)

DIAGRAM

```

+-----+
| DLinkedList |
+-----+
| key: int    | 1 * |
| value: int   |<----->|
| prev: DLinkedList |
| next: DLinkedList |
+-----+

+-----+
| LRUCache    |
+-----+
| capacity: int |
| cache: dict   |
| head: DLinkedList |
| tail: DLinkedList |
+-----+
| get(key) -> int |
| put(key, val)   |
| _remove(node)  |
| _add_to_front(n)|
+-----+

```

Design patterns used

- **No classic GoF pattern** — this is a pure data-structure problem. The key insight is the **sentinel head/tail** trick to avoid null checks. Interviewers want to see you know this.
- Mention: for thread-safety you'd wrap with a `threading.Lock` or use `collections.OrderedDict` (which Python's `stdlib` provides for a simpler implementation, but interviewers want the manual version to test understanding).

Full code

```

from __future__ import annotations
from typing import Optional

class DLinkedNode:
    """Doubly-linked list node used as the internal building block."""

    def __init__(self, key: int = 0, value: int = 0) → None:
        self.key = key
        self.value = value
        self.prev: Optional[DLinkedNode] = None
        self.next: Optional[DLinkedNode] = None

class LRUCache:
    """
    O(1) get/put LRU Cache implemented with a HashMap + doubly-linked list.

    Invariant: the list is ordered most-recently-used (front) to
    least-recently-used (back). Sentinel head/tail nodes eliminate
    edge-case null checks.
    """

    def __init__(self, capacity: int) → None:
        if capacity ≤ 0:
            raise ValueError("Capacity must be positive")
        self.capacity = capacity
        self.cache: dict[int, DLinkedNode] = {}

        # Sentinels - never evicted, never returned
        self.head = DLinkedNode() # MRU sentinel
        self.tail = DLinkedNode() # LRU sentinel
        self.head.next = self.tail
        self.tail.prev = self.head

    # ----- #
    # Public API                                     #
    # ----- #

    def get(self, key: int) → int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._move_to_front(node)
        return node.value

    def put(self, key: int, value: int) → None:
        if key in self.cache:
            node = self.cache[key]
            node.value = value
            self._move_to_front(node)
        else:
            node = DLinkedNode(key, value)
            self.cache[key] = node
            self._add_to_front(node)
            if len(self.cache) > self.capacity:
                lru = self._pop_from_back()
                del self.cache[lru.key]

    # ----- #
    # Private helpers                               #
    # ----- #

    def _add_to_front(self, node: DLinkedNode) → None:
        """Insert node right after head sentinel (= most-recently-used)."""
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def _remove(self, node: DLinkedNode) → None:
        """Unlink node from its current position."""
        node.prev.next = node.next
        node.next.prev = node.prev

```

```

def _move_to_front(self, node: DLinkedNode) → None:
    self._remove(node)
    self._add_to_front(node)

def _pop_from_back(self) → DLinkedNode:
    """Remove and return the LRU node (just before tail sentinel)."""
    lru = self.tail.prev
    self._remove(lru)
    return lru

# ----- #
# LRU variant - brief notes (full implementation would add freq map) #
# ----- #
# LRU Cache (Least Frequently Used):
# - Evict the entry with the LOWEST access frequency;
#   tie-break on LRU order within same frequency.
# Data structures needed:
# - key_to_val: dict[key → value]
# - key_to_freq: dict[key → frequency]
# - freq_to_keys: dict[freq → OrderedDict[key → None]] (ordered = LRU within freq)
# - min_freq: int (track current minimum)
# On get/put: increment freq, move key to freq+1 bucket, update min_freq.
# All operations O(1). See Python's collections.OrderedDict for ordered bucket.

if __name__ == "__main__":
    cache = LRUCache(capacity=3)
    cache.put(1, 10)
    cache.put(2, 20)
    cache.put(3, 30)

    print(cache.get(1)) # 10 - 1 is now MRU
    cache.put(4, 40) # evicts 2 (LRU)
    print(cache.get(2)) # -1 - evicted
    print(cache.get(3)) # 30
    cache.put(5, 50) # evicts 4 (LRU after 1 and 3 were accessed)
    print(cache.get(4)) # -1
    print(cache.get(1)) # 10
    print(cache.get(3)) # 30
    print(cache.get(5)) # 50

```

Extensibility discussion

- **TTL / expiry:** Add expiry: `Optional[float]` to `DLinkedNode`. On get, check `time.time() > expiry` and treat as miss + remove. A background thread or lazy expiry on access both work.
- **Thread-safety:** Wrap get/put with `threading.Lock`. For higher concurrency, shard the cache into `N` sub-caches keyed by `hash(key) % N`.
- **Generics:** Replace `int` with `TypeVar('K')` / `TypeVar('V')` and reuse across types.

Interview takeaway: The sentinel head/tail trick is the key insight. Without sentinels you need `if node.prev` null-checks everywhere — messy and error-prone. Name it explicitly.

2. Parking Lot

Requirements

Functional - Support multiple levels, multiple spot types (COMPACT, LARGE, MOTORCYCLE, ELECTRIC). - Vehicles: Motorcycle, Car, Truck — each fits specific spot types. - `park(vehicle)` — find nearest available spot; return ticket. - `unpark(ticket)` — free the spot; compute fee. - Pluggable pricing strategy (hourly, flat-rate, etc.). - Display available spots per level.

Clarifying questions to ask - Can a car park in a large spot if compact is full? (Yes, with preference ordering.) - Is pricing per hour or per entry? (Make it a strategy — both.) - Multiple entrances? (Assume single for now; mention extension.) - Reservations / pre-booking? (Out of scope.) - Thread-safety for concurrent park calls? (Mention lock per level; skip for now.)

Core entities / classes

Class	Responsibility
VehicleType (enum)	MOTORCYCLE, CAR, TRUCK
SpotType (enum)	MOTORCYCLE, COMPACT, LARGE, ELECTRIC
Vehicle (ABC)	Base vehicle with type + license plate
ParkingSpot	Individual spot — type, level, number, occupied flag
ParkingLevel	Collection of spots on one floor; find-spot logic
ParkingLot	Singleton; owns levels; exposes park/unpark
Ticket	Issued on park; holds spot ref + entry time
PricingStrategy (ABC)	Strategy interface: <code>calculate_fee(ticket) -> float</code>
HourlyPricing	Concrete strategy
FlatRatePricing	Concrete strategy

Class diagram (ASCII UML)

DIAGRAM

```

+-----+
| ParkingLot | <<Singleton>>
+-----+
| levels: List[Level]
| strategy: PricingStrategy
+-----+
| park(vehicle) |
| unpark(ticket) |
+-----+
| 1..*
+-----v-----+
| ParkingLevel |
+-----+
| level_num: int |
| spots: List[Spot] |
+-----+
| find_spot(vehicle)|
+-----+
| 1..*
+-----v-----+
| ParkingSpot |
+-----+
| spot_type: SpotType
| is_available: bool|
| vehicle: Vehicle |
+-----+

<<ABC>>
+-----+
| PricingStrategy |
+-----+
| calculate_fee() |
+-----+
|
+-----+
| |
+-----+ +-----+
|Hourly| |FlatRate|
+-----+ +-----+

<<ABC>>
+-----+
| Vehicle |
+-----+
| license: str |
| type: VehicleType|
+-----+
^
+-----+
| | |
| Moto | Car | Truck

```

Design patterns used

- **Strategy** — PricingStrategy lets you swap hourly/flat-rate/weekend pricing without touching ParkingLot.
- **Singleton** — ParkingLot represents a single physical lot; use `__new__` or a module-level instance.
- **Factory Method** — VehicleFactory.create(type, plate) to instantiate vehicles without if/else in client code.
- **Template Method** — Vehicle defines allowed_spot_types() as an abstract method; each subclass overrides it.

Full code

```

from __future__ import annotations

import time
from abc import ABC, abstractmethod
from enum import Enum, auto
from typing import Optional
from dataclasses import dataclass, field

# ----- #
# Enums                                     #
# ----- #

class VehicleType(Enum):
    MOTORCYCLE = auto()
    CAR = auto()
    TRUCK = auto()

class SpotType(Enum):
    MOTORCYCLE = auto()
    COMPACT = auto()
    LARGE = auto()
    ELECTRIC = auto()

# ----- #
# Vehicles                                 #
# ----- #

class Vehicle(ABC):
    def __init__(self, license_plate: str, vehicle_type: VehicleType) → None:
        self.license_plate = license_plate
        self.vehicle_type = vehicle_type

    @abstractmethod
    def allowed_spot_types(self) → list[SpotType]:
        """Ordered preference: first = most preferred spot type."""
        ...

    def __repr__(self) → str:
        return f"{self.__class__.__name__}({self.license_plate})"

class Motorcycle(Vehicle):
    def __init__(self, license_plate: str) → None:
        super().__init__(license_plate, VehicleType.MOTORCYCLE)

    def allowed_spot_types(self) → list[SpotType]:
        return [SpotType.MOTORCYCLE, SpotType.COMPACT, SpotType.LARGE]

class Car(Vehicle):
    def __init__(self, license_plate: str) → None:
        super().__init__(license_plate, VehicleType.CAR)

    def allowed_spot_types(self) → list[SpotType]:
        return [SpotType.COMPACT, SpotType.LARGE, SpotType.ELECTRIC]

class Truck(Vehicle):
    def __init__(self, license_plate: str) → None:
        super().__init__(license_plate, VehicleType.TRUCK)

    def allowed_spot_types(self) → list[SpotType]:
        return [SpotType.LARGE]

class VehicleFactory:
    """Factory to decouple client from concrete vehicle classes."""

    _registry: dict[VehicleType, type[Vehicle]] = {
        VehicleType.MOTORCYCLE: Motorcycle,
        VehicleType.CAR: Car,

```

```

        VehicleType.TRUCK: Truck,
    }

    @classmethod
    def create(cls, vehicle_type: VehicleType, license_plate: str) → Vehicle:
        klass = cls._registry.get(vehicle_type)
        if klass is None:
            raise ValueError(f"Unknown vehicle type: {vehicle_type}")
        return klass(license_plate)

# ----- #
# Parking Spot                                     #
# ----- #

class ParkingSpot:
    def __init__(self, spot_id: str, spot_type: SpotType, level: int) → None:
        self.spot_id = spot_id
        self.spot_type = spot_type
        self.level = level
        self._vehicle: Optional[Vehicle] = None

    @property
    def is_available(self) → bool:
        return self._vehicle is None

    def park(self, vehicle: Vehicle) → None:
        if not self.is_available:
            raise RuntimeError(f"Spot {self.spot_id} already occupied")
        self._vehicle = vehicle

    def unpark(self) → Vehicle:
        if self.is_available:
            raise RuntimeError(f"Spot {self.spot_id} is empty")
        vehicle = self._vehicle
        self._vehicle = None
        return vehicle # type: ignore[return-value]

    def __repr__(self) → str:
        status = "FREE" if self.is_available else f"OCC({self._vehicle})"
        return f"Spot[{self.spot_id},{self.spot_type.name},{status}]"

# ----- #
# Ticket                                           #
# ----- #

@dataclass
class Ticket:
    ticket_id: str
    vehicle: Vehicle
    spot: ParkingSpot
    entry_time: float = field(default_factory=time.time)
    exit_time: Optional[float] = None

    def duration_hours(self) → float:
        end = self.exit_time or time.time()
        return (end - self.entry_time) / 3600.0

# ----- #
# Pricing Strategy                               #
# ----- #

class PricingStrategy(ABC):
    @abstractmethod
    def calculate_fee(self, ticket: Ticket) → float:
        ...

class HourlyPricing(PricingStrategy):
    def __init__(self, rate_per_hour: float = 2.0) → None:
        self.rate_per_hour = rate_per_hour

    def calculate_fee(self, ticket: Ticket) → float:
        import math
        hours = math.ceil(ticket.duration_hours()) # round up to next hour

```

```

        return max(hours, 1) * self.rate_per_hour

class FlatRatePricing(PricingStrategy):
    def __init__(self, flat_fee: float = 5.0) → None:
        self.flat_fee = flat_fee

    def calculate_fee(self, ticket: Ticket) → float:
        return self.flat_fee

# ----- #
# Parking Level                                     #
# ----- #

class ParkingLevel:
    def __init__(self, level_num: int, spots: list[ParkingSpot]) → None:
        self.level_num = level_num
        self.spots = spots

    def find_spot(self, vehicle: Vehicle) → Optional[ParkingSpot]:
        """Return first available spot matching vehicle preference order."""
        for preferred_type in vehicle.allowed_spot_types():
            for spot in self.spots:
                if spot.spot_type == preferred_type and spot.is_available:
                    return spot
        return None

    def available_count(self) → dict[SpotType, int]:
        counts: dict[SpotType, int] = {}
        for spot in self.spots:
            if spot.is_available:
                counts[spot.spot_type] = counts.get(spot.spot_type, 0) + 1
        return counts

# ----- #
# Parking Lot (Singleton)                           #
# ----- #

class ParkingLot:
    _instance: Optional[ParkingLot] = None

    def __new__(cls, *args, **kwargs) → ParkingLot:
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(
        self,
        name: str,
        levels: list[ParkingLevel],
        pricing_strategy: Optional[PricingStrategy] = None,
    ) → None:
        # Guard against re-init on subsequent __new__ returns
        if hasattr(self, "_initialized"):
            return
        self.name = name
        self.levels = levels
        self.pricing_strategy: PricingStrategy = pricing_strategy or HourlyPricing()
        self._tickets: dict[str, Ticket] = {}
        self._ticket_counter = 0
        self._initialized = True

    def park(self, vehicle: Vehicle) → Ticket:
        for level in self.levels:
            spot = level.find_spot(vehicle)
            if spot:
                spot.park(vehicle)
                self._ticket_counter += 1
                ticket = Ticket(
                    ticket_id=f"T{self._ticket_counter:04d}",
                    vehicle=vehicle,
                    spot=spot,
                )
                self._tickets[ticket.ticket_id] = ticket
        print(f"  Parked {vehicle} at {spot.spot_id} (Level {level.level_num})")

```

```

        return ticket
    raise RuntimeError("No available spot for this vehicle type")

def unpark(self, ticket_id: str) → float:
    ticket = self._tickets.get(ticket_id)
    if ticket is None:
        raise ValueError(f"Unknown ticket: {ticket_id}")
    ticket.exit_time = time.time()
    ticket.spot.unpark()
    fee = self.pricing_strategy.calculate_fee(ticket)
    del self._tickets[ticket_id]
    print(f"  Unparked {ticket.vehicle} | Fee: ${fee:.2f}")
    return fee

def display_availability(self) → None:
    print(f"\n--- {self.name} Availability ---")
    for level in self.levels:
        counts = level.available_count()
        print(f"  Level {level.level_num}: {counts}")

# ----- #
# Helper: build a lot
# ----- #

def build_parking_lot() → ParkingLot:
    # Reset singleton for demo purposes
    ParkingLot._instance = None

    def make_spots(level: int, config: dict[SpotType, int]) → list[ParkingSpot]:
        spots = []
        for spot_type, count in config.items():
            for i in range(1, count + 1):
                spot_id = f"L{level}-{spot_type.name[0]}{i}"
                spots.append(ParkingSpot(spot_id, spot_type, level))
        return spots

    levels = [
        ParkingLevel(1, make_spots(1, {
            SpotType.MOTORCYCLE: 5,
            SpotType.COMPACT: 10,
            SpotType.LARGE: 5,
        })),
        ParkingLevel(2, make_spots(2, {
            SpotType.COMPACT: 8,
            SpotType.LARGE: 4,
            SpotType.ELECTRIC: 3,
        })),
    ]
    return ParkingLot("City Center Garage", levels, HourlyPricing(rate_per_hour=3.0))

if __name__ == "__main__":
    lot = build_parking_lot()
    lot.display_availability()

    moto = Motorcycle("MOTO-001")
    car1 = Car("CAR-001")
    truck = Truck("TRK-001")

    t1 = lot.park(moto)
    t2 = lot.park(car1)
    t3 = lot.park(truck)
    lot.display_availability()

    # Simulate time passing (normally you'd actually wait)
    t2.entry_time -= 7200 # pretend car parked 2 hours ago
    lot.unpark(t2.ticket_id)
    lot.unpark(t1.ticket_id)
    lot.display_availability()

```

Extensibility discussion

- **New vehicle type (e.g., Bus):** Add `VehicleType.BUS`, create class `Bus(Vehicle)` with `allowed_spot_types`, register in `VehicleFactory._registry`. Zero changes to `ParkingLot` or `ParkingLevel`.

- **New pricing (weekend/dynamic):** Implement `WeekendPricing(PricingStrategy)`. Pass it at construction. OCP satisfied.
- **EV charging:** Add `SpotType.ELECTRIC`; `Car.allowed_spot_types` already supports it. Add `ChargeSession` attached to `Ticket`.
- **Reservations:** Add `reserve(vehicle, start_time)` that marks a spot with a future timestamp; `find_spot` skips reserved spots for the overlapping window.

Interview takeaway: Always name the Strategy pattern for pricing — it's the most common follow-up ("what if pricing changes on weekends?"). The answer is: new class, zero changes to existing code.

3. Rate Limiter

Requirements

Functional - `allow(client_id) -> bool` — return True if request should be permitted. - Support Token Bucket, Sliding Window Log, Fixed Window Counter strategies. - Pluggable — caller picks strategy at construction. - Per-client limiting (different clients independent).

Clarifying questions to ask - Global limit or per-client? (Per-client.) - Burst tolerance? (Token Bucket allows bursts; fixed-window does not.) - Distributed (Redis-backed)? (Assume in-process for now; mention Redis extension.) - What happens on limit hit — queue or reject? (Reject / return False.) - Thread-safe? (Yes, per-client locks.)

Core entities / classes

Class	Responsibility
RateLimitStrategy (ABC)	Interface: <code>is_allowed(client_id) -> bool</code>
TokenBucketStrategy	Refill tokens at fixed rate; allow if tokens > 0
FixedWindowStrategy	Count requests per fixed time window; reset on window roll
SlidingWindowLogStrategy	Keep timestamped log; count entries in last N seconds
RateLimiter	Context object; delegates to strategy

Class diagram (ASCII UML)

DIAGRAM

```

+-----+
|   RateLimiter   |
+-----+
| strategy: RateLimitStrategy
+-----+
| allow(client_id)→bool|
+-----+
|
| uses
+-----v-----+
| RateLimitStrategy |<--<<ABC>>+
+-----+
| is_allowed(cid)→bool|
+-----+
|
|
+-----+
| TokenBucket | FixedWindow | SlidingWindowLog
+-----+

```

Design patterns used

- **Strategy** — core pattern; swap algorithms at runtime.
- **Factory / constructor injection** — `RateLimiter(strategy=TokenBucketStrategy(...))`.

Full code

```

from __future__ import annotations

import time
import threading
from abc import ABC, abstractmethod
from collections import deque
from dataclasses import dataclass, field
from typing import Optional

# ----- #
# Strategy Interface                                     #
# ----- #

class RateLimitStrategy(ABC):
    @abstractmethod
    def is_allowed(self, client_id: str) → bool:
        ...

# ----- #
# Strategy 1: Token Bucket                               #
# ----- #

@dataclass
class _TokenBucketState:
    tokens: float
    last_refill: float = field(default_factory=time.time)

class TokenBucketStrategy(RateLimitStrategy):
    """
    Each client starts with `capacity` tokens.
    Tokens refill at `refill_rate` tokens/second.
    Each request consumes 1 token. Allows bursts up to `capacity`.
    """

    def __init__(self, capacity: float, refill_rate: float) → None:
        self.capacity = capacity
        self.refill_rate = refill_rate
        self._buckets: dict[str, _TokenBucketState] = {}
        self._lock = threading.Lock()

    def _get_bucket(self, client_id: str) → _TokenBucketState:
        if client_id not in self._buckets:
            self._buckets[client_id] = _TokenBucketState(tokens=self.capacity)
        return self._buckets[client_id]

    def is_allowed(self, client_id: str) → bool:
        with self._lock:
            now = time.time()
            bucket = self._get_bucket(client_id)
            elapsed = now - bucket.last_refill
            bucket.tokens = min(
                self.capacity,
                bucket.tokens + elapsed * self.refill_rate
            )
            bucket.last_refill = now
            if bucket.tokens ≥ 1:
                bucket.tokens -= 1
                return True
            return False

# ----- #
# Strategy 2: Fixed Window Counter                       #
# ----- #

@dataclass
class _FixedWindowState:
    count: int = 0
    window_start: float = field(default_factory=time.time)

```

```

class FixedWindowStrategy(RateLimitStrategy):
    """
    Allow up to `limit` requests per `window_seconds` window.
    Window resets at the start of each period.
    Weakness: burst at window boundary (2x limit in 2*epsilon seconds).
    """

    def __init__(self, limit: int, window_seconds: float) → None:
        self.limit = limit
        self.window_seconds = window_seconds
        self._states: dict[str, _FixedWindowState] = {}
        self._lock = threading.Lock()

    def is_allowed(self, client_id: str) → bool:
        with self._lock:
            now = time.time()
            if client_id not in self._states:
                self._states[client_id] = _FixedWindowState(window_start=now)
            state = self._states[client_id]

            if now - state.window_start ≥ self.window_seconds:
                state.count = 0
                state.window_start = now

            if state.count < self.limit:
                state.count += 1
                return True
            return False

# ----- #
# Strategy 3: Sliding Window Log                                     #
# ----- #

class SlidingWindowLogStrategy(RateLimitStrategy):
    """
    Keep a deque of request timestamps per client.
    On each request, prune entries older than `window_seconds`,
    then check if remaining count < limit.
    Most accurate; memory = O(limit) per client.
    """

    def __init__(self, limit: int, window_seconds: float) → None:
        self.limit = limit
        self.window_seconds = window_seconds
        self._logs: dict[str, deque[float]] = {}
        self._lock = threading.Lock()

    def is_allowed(self, client_id: str) → bool:
        with self._lock:
            now = time.time()
            if client_id not in self._logs:
                self._logs[client_id] = deque()
            log = self._logs[client_id]

            cutoff = now - self.window_seconds
            while log and log[0] < cutoff:
                log.popleft()

            if len(log) < self.limit:
                log.append(now)
                return True
            return False

# ----- #
# Rate Limiter Context                                           #
# ----- #

class RateLimiter:
    """
    Context class. Delegates to the injected strategy.
    Swap strategy at runtime with `set_strategy`.
    """

    def __init__(self, strategy: RateLimitStrategy) → None:
        self._strategy = strategy

```

```

def set_strategy(self, strategy: RateLimitStrategy) → None:
    self._strategy = strategy

def allow(self, client_id: str) → bool:
    return self._strategy.is_allowed(client_id)

# ----- #
# Demo                                         #
# ----- #

if __name__ == "__main__":
    print("=== Token Bucket (capacity=5, refill=1/s) ===")
    rl = RateLimiter(TokenBucketStrategy(capacity=5, refill_rate=1.0))
    for i in range(7):
        result = rl.allow("user_A")
        print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")

    print("\n=== Fixed Window (limit=3, window=10s) ===")
    rl2 = RateLimiter(FixedWindowStrategy(limit=3, window_seconds=10))
    for i in range(5):
        result = rl2.allow("user_B")
        print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")

    print("\n=== Sliding Window Log (limit=3, window=1s) ===")
    rl3 = RateLimiter(SlidingWindowLogStrategy(limit=3, window_seconds=1.0))
    for i in range(4):
        result = rl3.allow("user_C")
        print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")
    time.sleep(1.1)
    result = rl3.allow("user_C")
    print(f" After sleep: {'ALLOWED' if result else 'DENIED'}")

```

Extensibility discussion

- **Redis-backed distributed limiter:** Replace the in-memory `_buckets/_logs` dicts with Redis calls using atomic INCR/EXPIRE or Lua scripts. Strategy interface unchanged.
- **New algorithm (leaky bucket):** Implement `LeakyBucketStrategy(RateLimitStrategy)`. Inject it. No other changes.
- **Middleware integration:** Wrap `RateLimiter.allow` in a decorator or WSGI/ASGI middleware. The core class is unaware of HTTP.
- **Per-endpoint limits:** Key on `f"{client_id}:{endpoint}"` instead of just `client_id`.

Interview takeaway: Know the trade-offs: Token Bucket allows bursts; Fixed Window has boundary-burst weakness; Sliding Window Log is most accurate but costs $O(\text{limit})$ memory per client. Sliding Window Counter (hybrid) is the production sweet spot.

4. Splitwise / Expense Sharing

Requirements

Functional - Add users to groups. - Add an expense: payer, amount, split type (EQUAL, EXACT, PERCENT), participants. - show_balances(user) — who owes whom and how much. - simplify_debts(group) — minimize number of transactions to settle.

Clarifying questions to ask - Currency? (Single currency; mention multi-currency extension.) - Can the payer be a participant? (Yes — they effectively owe zero or receive money.) - Simplify debts: across whole app or per-group? (Per-group for now.) - Partial payments / settle-up? (Yes — add expense with type = SETTLE.) - Persistence? (In-memory; mention DB extension.)

Core entities / classes

Class	Responsibility
User	Name, id, personal balance map
Group	Collection of users + expenses
Expense	Amount, payer, participants, split strategy
SplitStrategy (ABC)	compute_shares(amount, participants) -> dict[User, float]
EqualSplit	Divide evenly
ExactSplit	Caller specifies each person's share
PercentSplit	Caller specifies each person's percentage
Balance	Utility: compute net balances + simplify

Class diagram (ASCII UML)

DIAGRAM

```

+-----+      +-----+      +-----+
|  User  |      |  Group  |      | Expense |
+-----+      +-----+      +-----+
| id: str |<---->| users  |<---->| amount  |
| name: str|      | expenses|      | payer: User|
+-----+      +-----+      +-----+
                                   | split: Split
                                   +-----+
                                   | uses
                                   +-----+
                                   <<ABC>> +---V-----+
                                   +-----+|SplitStrat|
                                   | Equal | +-----+
                                   | Exact  || compute() |
                                   | Percent| +-----+
                                   +-----+

```

Design patterns used

- **Strategy** — SplitStrategy for different split computations.
- **Domain Model** — User, Group, Expense map directly to business entities.

Full code

```

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum, auto
from typing import Optional
import uuid

# ----- #
# Split Strategies                                     #
# ----- #

class SplitStrategy(ABC):
    @abstractmethod
    def compute_shares(
        self, amount: float, participants: list[User], **kwargs
    ) → dict[str, float]:
        """Return {user_id: share_amount} for each participant."""
        ...

class EqualSplit(SplitStrategy):
    def compute_shares(
        self, amount: float, participants: list[User], **kwargs
    ) → dict[str, float]:
        share = round(amount / len(participants), 2)
        result = {u.user_id: share for u in participants}
        # Correct rounding error on first participant
        total = sum(result.values())
        diff = round(amount - total, 2)
        first = participants[0].user_id
        result[first] = round(result[first] + diff, 2)
        return result

class ExactSplit(SplitStrategy):
    def compute_shares(
        self, amount: float, participants: list[User], **kwargs
    ) → dict[str, float]:
        shares: dict[str, float] = kwargs.get("exact_shares", {})
        if abs(sum(shares.values()) - amount) > 0.01:
            raise ValueError("Exact shares must sum to total amount")
        return {u.user_id: shares[u.user_id] for u in participants}

class PercentSplit(SplitStrategy):
    def compute_shares(
        self, amount: float, participants: list[User], **kwargs
    ) → dict[str, float]:
        percents: dict[str, float] = kwargs.get("percents", {})
        if abs(sum(percents.values()) - 100) > 0.01:
            raise ValueError("Percentages must sum to 100")
        return {
            u.user_id: round(amount * percents[u.user_id] / 100, 2)
            for u in participants
        }

# ----- #
# User                                               #
# ----- #

@dataclass
class User:
    name: str
    user_id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])

    def __hash__(self) → int:
        return hash(self.user_id)

    def __eq__(self, other: object) → bool:
        return isinstance(other, User) and self.user_id == other.user_id

```

```

def __repr__(self) → str:
    return f"User({self.name})"

# ----- #
# Expense #
# ----- #

class SplitType(Enum):
    EQUAL = auto()
    EXACT = auto()
    PERCENT = auto()

@dataclass
class Expense:
    expense_id: str
    description: str
    amount: float
    payer: User
    participants: list[User]
    split_strategy: SplitStrategy
    shares: dict[str, float] # user_id → amount owed by that user

# ----- #
# Group #
# ----- #

class Group:
    def __init__(self, name: str) → None:
        self.group_id = str(uuid.uuid4())[:8]
        self.name = name
        self.members: list[User] = []
        self.expenses: list[Expense] = []
        # net_balances[a][b] = amount b owes a (positive = b owes a)
        self._net: dict[str, dict[str, float]] = {}

    def add_member(self, user: User) → None:
        if user not in self.members:
            self.members.append(user)
            self._net.setdefault(user.user_id, {})

    def add_expense(
        self,
        description: str,
        amount: float,
        payer: User,
        participants: list[User],
        strategy: SplitStrategy,
        **kwargs,
    ) → Expense:
        if payer not in self.members:
            raise ValueError(f"{payer} is not a group member")
        shares = strategy.compute_shares(amount, participants, **kwargs)
        expense = Expense(
            expense_id=str(uuid.uuid4())[:8],
            description=description,
            amount=amount,
            payer=payer,
            participants=participants,
            split_strategy=strategy,
            shares=shares,
        )
        self.expenses.append(expense)
        self._update_balances(expense)
        return expense

    def _update_balances(self, expense: Expense) → None:
        payer_id = expense.payer.user_id
        for uid, share in expense.shares.items():
            if uid == payer_id:
                continue # payer doesn't owe themselves
            # uid owes payer_id `share` amount
            self._net.setdefault(payer_id, {})
            self._net.setdefault(uid, {})
            current = self._net[payer_id].get(uid, 0)

```

```

        self._net[payer_id][uid] = round(current + share, 2)

def show_balances(self) → None:
    print(f"\n--- Balances in group '{self.name}' ---")
    any_balance = False
    for creditor_id, debtors in self._net.items():
        creditor_name = self._user_name(creditor_id)
        for debtor_id, amount in debtors.items():
            net = amount - self._net.get(debtor_id, {}).get(creditor_id, 0)
            if net > 0.01:
                any_balance = True
                debtor_name = self._user_name(debtor_id)
                print(f" {debtor_name} owes {creditor_name}: ${net:.2f}")
    if not any_balance:
        print(" All settled up!")

def simplify_debts(self) → list[tuple[str, str, float]]:
    """
    Minimize number of transactions using a greedy creditor-debtor matching.
    Returns list of (debtor_name, creditor_name, amount).
    """
    # Compute net position for each user
    net_position: dict[str, float] = {m.user_id: 0.0 for m in self.members}
    for creditor_id, debtors in self._net.items():
        for debtor_id, amount in debtors.items():
            net_position[creditor_id] = round(net_position[creditor_id] + amount, 2)
            net_position[debtor_id] = round(net_position[debtor_id] - amount, 2)

    # Separate into creditors (positive) and debtors (negative)
    creditors = {uid: amt for uid, amt in net_position.items() if amt > 0.01}
    debtors = {uid: -amt for uid, amt in net_position.items() if amt < -0.01}

    transactions: list[tuple[str, str, float]] = []
    cred_list = sorted(creditors.items(), key=lambda x: -x[1])
    debt_list = sorted(debtors.items(), key=lambda x: -x[1])

    i, j = 0, 0
    while i < len(cred_list) and j < len(debt_list):
        cid, c_amt = cred_list[i]
        did, d_amt = debt_list[j]
        settled = round(min(c_amt, d_amt), 2)
        transactions.append((self._user_name(did), self._user_name(cid), settled))
        c_amt = round(c_amt - settled, 2)
        d_amt = round(d_amt - settled, 2)
        cred_list[i] = (cid, c_amt)
        debt_list[j] = (did, d_amt) # dummy; just advance index
        if c_amt < 0.01:
            i += 1
        if d_amt < 0.01:
            j += 1

    return transactions

def _user_name(self, user_id: str) → str:
    for m in self.members:
        if m.user_id == user_id:
            return m.name
    return user_id

if __name__ == "__main__":
    alice = User("Alice")
    bob = User("Bob")
    charlie = User("Charlie")

    group = Group("Trip to Goa")
    for u in [alice, bob, charlie]:
        group.add_member(u)

    # Alice pays Rs 300 split equally
    group.add_expense(
        "Dinner", 300.0, alice, [alice, bob, charlie], EqualSplit()
    )

    # Bob pays Rs 200: Charlie pays 50%, Bob 30%, Alice 20%
    group.add_expense(
        "Cab", 200.0, bob,

```

```

    [alice, bob, charlie], PercentSplit(),
    percents={alice.user_id: 20, bob.user_id: 30, charlie.user_id: 50}
)

group.show_balances()

print("\n--- Simplified transactions ---")
for debtor, creditor, amount in group.simplify_debts():
    print(f" {debtor} pays {creditor}: ${amount:.2f}")

```

Extensibility discussion

- **New split type (shares-based):** Add `SharesSplit(SplitStrategy)` where each participant has N shares; their fraction = $N_i / \text{sum}(N)$. Zero changes to `Group`.
- **Multi-currency:** Add currency: `str` to `Expense`; convert to a base currency using an injected `CurrencyConverter` before updating balances.
- **Settle-up:** `settle(payer, payee, amount)` is just `add_expense("Settlement", amount, payer, [payer, payee], ExactSplit(), exact_shares={payer.user_id:0, payee.user_id:amount})`.
- **Persistence:** Replace `self._net` updates with DB writes. The `Group` class becomes a service layer calling a repository.

Interview takeaway: The debt-simplification algorithm is a classic greedy: compute net positions, then greedily match biggest creditor with biggest debtor. It's $O(N \log N)$ and minimizes the number of transactions (though not necessarily the amounts).

5. Elevator System

Requirements

Functional - Multiple elevators in a building. - External requests: `request_floor(floor, direction)` — someone on floor 3 wants to go UP. - Internal requests: `select_floor(elevator_id, floor)` — someone inside presses a button. - Scheduler assigns best elevator to each external request. - Each elevator has states: IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE.

Clarifying questions to ask - How many elevators / floors? (Configurable.) - Any elevator capacity / weight limit? (Mention, skip implementation.) - Priority floors (emergency)? (Out of scope for now.) - Which scheduling algorithm? (SCAN / "look" algorithm — service in current direction first.) - Real-time simulation or just state transitions? (State transitions.)

Core entities / classes

Class	Responsibility
ElevatorState (enum)	IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE
Direction (enum)	UP, DOWN, IDLE
Request	Floor + direction (external) or just floor (internal)
Elevator	Single car; state machine; queue of floors to serve
Scheduler (ABC)	Assigns external request to an elevator
ScanScheduler	SCAN/LOOK algorithm implementation
ElevatorSystem	Owns elevators + scheduler; top-level API

Class diagram (ASCII UML)

DIAGRAM

```

+-----+
| ElevatorSystem |
+-----+
| elevators: list |
| scheduler: Scheduler |
+-----+
| request(floor, dir) |
| select_floor(eid,f) |
| step() |
+-----+
|
+-----+
| Elevator |
+-----+
| id: int |
| floor: int |
| state: State |
| destinations |
+-----+
|
+-----+
| Scheduler |
+-----+
| assign() |
+-----+
|
+-----+
| ScanSched |
+-----+

```

Design patterns used

- **State** — ElevatorState drives behavior. The transition table (IDLE → MOVING_UP, etc.) is explicit.
- **Strategy** — Scheduler is pluggable (SCAN, Nearest-Car, Zone-based).
- **Observer** (mention) — In production, elevators would publish state-change events; floors would subscribe to update displays.

Full code

```

from __future__ import annotations

from abc import ABC, abstractmethod
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Optional
import heapq

# ----- #
# Enums                                     #
# ----- #

class ElevatorState(Enum):
    IDLE = auto()
    MOVING_UP = auto()
    MOVING_DOWN = auto()
    MAINTENANCE = auto()

class Direction(Enum):
    UP = auto()
    DOWN = auto()
    IDLE = auto()

# ----- #
# Request                                   #
# ----- #

@dataclass
class Request:
    floor: int
    direction: Direction # Direction.IDLE for internal (in-elevator) requests
    is_internal: bool = False

    def __repr__(self) → str:
        kind = "INT" if self.is_internal else f"EXT-{self.direction.name}"
        return f"Req(floor={self.floor},{kind})"

# ----- #
# Elevator                                 #
# ----- #

class Elevator:
    """
    Single elevator car.

    Internally maintains two min-heaps:
    - up_queue: floors > current floor, to serve while going up (min-heap)
    - down_queue: floors < current floor, to serve while going down (max-heap via negation)
    SCAN algorithm: exhaust current direction first, then reverse.
    """

    def __init__(self, elevator_id: int, initial_floor: int = 1) → None:
        self.elevator_id = elevator_id
        self.current_floor = initial_floor
        self.state = ElevatorState.IDLE
        self.direction = Direction.IDLE
        self._up_queue: list[int] = [] # min-heap
        self._down_queue: list[int] = [] # max-heap (store negatives)

    def add_destination(self, floor: int) → None:
        if self.state == ElevatorState.MAINTENANCE:
            raise RuntimeError(f"Elevator {self.elevator_id} is under maintenance")
        if floor == self.current_floor:
            return
        if floor > self.current_floor:
            if floor not in self._up_queue:
                heapq.heappush(self._up_queue, floor)
        else:
            if -floor not in self._down_queue:
                heapq.heappush(self._down_queue, -floor)

```

```

self._update_state()

def step(self) → Optional[int]:
    """
    Move one floor toward the next destination.
    Returns the floor reached, or None if already at destination / idle.
    """
    if self.state == ElevatorState.IDLE or self.state == ElevatorState.MAINTENANCE:
        return None

    if self.state == ElevatorState.MOVING_UP:
        if self._up_queue:
            next_floor = self._up_queue[0]
            self.current_floor += 1
            print(f" Elevator {self.elevator_id} ↑ Floor {self.current_floor}")
            if self.current_floor == next_floor:
                heapq.heappop(self._up_queue)
                print(f" Elevator {self.elevator_id} STOP at Floor {self.current_floor}")
            self._update_state()

        elif self.state == ElevatorState.MOVING_DOWN:
            if self._down_queue:
                next_floor = -self._down_queue[0]
                self.current_floor -= 1
                print(f" Elevator {self.elevator_id} ↓ Floor {self.current_floor}")
                if self.current_floor == next_floor:
                    heapq.heappop(self._down_queue)
                    print(f" Elevator {self.elevator_id} STOP at Floor {self.current_floor}")
                self._update_state()

    return self.current_floor

def _update_state(self) → None:
    """Transition state based on queues and current direction."""
    if not self._up_queue and not self._down_queue:
        self.state = ElevatorState.IDLE
        self.direction = Direction.IDLE
    elif self.direction == Direction.UP:
        if self._up_queue:
            self.state = ElevatorState.MOVING_UP
        else:
            # Exhausted upward requests; switch down
            self.direction = Direction.DOWN
            self.state = ElevatorState.MOVING_DOWN
    elif self.direction == Direction.DOWN:
        if self._down_queue:
            self.state = ElevatorState.MOVING_DOWN
        else:
            self.direction = Direction.UP
            self.state = ElevatorState.MOVING_UP
    else:
        # Was IDLE; pick direction based on first pending request
        if self._up_queue and (
            not self._down_queue or self._up_queue[0] ≥ self.current_floor
        ):
            self.direction = Direction.UP
            self.state = ElevatorState.MOVING_UP
        else:
            self.direction = Direction.DOWN
            self.state = ElevatorState.MOVING_DOWN

def cost_to_serve(self, floor: int, direction: Direction) → int:
    """
    Heuristic cost for scheduler: number of steps to reach 'floor'.
    Considers current direction and queued stops.
    """
    if self.state == ElevatorState.MAINTENANCE:
        return float("inf") # type: ignore[return-value]

    if self.state == ElevatorState.IDLE:
        return abs(self.current_floor - floor)

    if self.state == ElevatorState.MOVING_UP:
        if floor ≥ self.current_floor and direction == Direction.UP:
            return floor - self.current_floor
        # Must finish upward, come back
        top = max(self._up_queue) if self._up_queue else self.current_floor

```

```

        return (top - self.current_floor) + (top - floor)

    if self.state == ElevatorState.MOVING_DOWN:
        if floor ≤ self.current_floor and direction == Direction.DOWN:
            return self.current_floor - floor
        bottom = min(-x for x in self._down_queue) if self._down_queue else self.current_floor
        return (self.current_floor - bottom) + (floor - bottom)

    return abs(self.current_floor - floor)

def __repr__(self) → str:
    return (
        f"Elevator[{self.elevator_id}]Floor {self.current_floor}|"
        f"{self.state.name}|up={list(self._up_queue)}|"
        f"down={[-x for x in self._down_queue]}"
    )

# ----- #
# Scheduler                                     #
# ----- #

class Scheduler(ABC):
    @abstractmethod
    def assign(self, elevators: list[Elevator], request: Request) → Elevator:
        ...

class NearestCarScheduler(Scheduler):
    """Assign the elevator with the lowest cost_to_serve heuristic."""

    def assign(self, elevators: list[Elevator], request: Request) → Elevator:
        available = [e for e in elevators if e.state ≠ ElevatorState.MAINTENANCE]
        if not available:
            raise RuntimeError("No elevators available")
        return min(
            available,
            key=lambda e: e.cost_to_serve(request.floor, request.direction)
        )

# ----- #
# Elevator System                             #
# ----- #

class ElevatorSystem:
    def __init__(
        self,
        num_elevators: int,
        num_floors: int,
        scheduler: Optional[Scheduler] = None,
    ) → None:
        self.num_floors = num_floors
        self.elevators = [Elevator(i + 1) for i in range(num_elevators)]
        self.scheduler = scheduler or NearestCarScheduler()

    def external_request(self, floor: int, direction: Direction) → None:
        """Someone on `floor` pressed the UP or DOWN button."""
        self._validate_floor(floor)
        request = Request(floor=floor, direction=direction)
        elevator = self.scheduler.assign(self.elevators, request)
        elevator.add_destination(floor)
        print(f" Assigned {request} to Elevator {elevator.elevator_id}")

    def internal_request(self, elevator_id: int, floor: int) → None:
        """Passenger inside elevator presses button for `floor`."""
        self._validate_floor(floor)
        elevator = self._get_elevator(elevator_id)
        elevator.add_destination(floor)
        print(f" Elevator {elevator_id}: internal request for floor {floor}")

    def step_all(self, steps: int = 1) → None:
        """Simulate `steps` time ticks."""
        for _ in range(steps):
            for elevator in self.elevators:
                elevator.step()

```

```

def status(self) → None:
    print("\n--- Elevator Status ---")
    for e in self.elevators:
        print(f" {e}")

def _validate_floor(self, floor: int) → None:
    if not (1 ≤ floor ≤ self.num_floors):
        raise ValueError(f"Floor {floor} out of range [1, {self.num_floors}]")

def _get_elevator(self, elevator_id: int) → Elevator:
    for e in self.elevators:
        if e.elevator_id == elevator_id:
            return e
    raise ValueError(f"Elevator {elevator_id} not found")

if __name__ == "__main__":
    system = ElevatorSystem(num_elevators=2, num_floors=10)
    system.status()

    print("\n--- Requests ---")
    system.external_request(5, Direction.UP) # person on floor 5 wants to go up
    system.external_request(3, Direction.DOWN) # person on floor 3 wants to go down
    system.internal_request(1, 8) # inside elevator 1, press 8

    system.status()

    print("\n--- Simulating 10 steps ---")
    system.step_all(steps=10)
    system.status()

```

Extensibility discussion

- **New scheduler (zone-based):** Implement `ZoneScheduler(Scheduler)` that restricts each elevator to a floor range. Inject at construction.
- **Emergency mode:** Add `ElevatorState.EMERGENCY`. Override `_update_state` to route all cars to ground floor first.
- **Capacity tracking:** Add `current_load: int` and `max_capacity: int` to `Elevator`. Scheduler skips full elevators.
- **Floor display / observer:** Elevators publish `FloorReachedEvent`; `FloorDisplay` objects subscribe and update arrow indicators.

Interview takeaway: The **SCAN algorithm** (also called "elevator algorithm") is exactly how real elevator schedulers work — it's the same algorithm used by disk schedulers. Name it. The state machine with direction + two queues (up/down) is the key implementation insight.

6. Tic-Tac-Toe (Clean OO Board Game)

Requirements

Functional - Two players, configurable board size (default 3×3). - `make_move(player, row, col)` — place mark; validate; check win/draw. - Win condition: any row, column, or diagonal fully owned by one player. - Draw: board full, no winner. - Support pluggable players (human, AI/random).

Clarifying questions to ask - Board size always 3×3 or configurable? (Configurable; win requires N-in-a-row.) - Two players only, or more? (Two for now; mention extension.) - AI player? (Optional; use Strategy.) - Can the same player go twice? (No — enforce turn order.) - Replay / undo moves? (Out of scope; mention Command pattern for undo.)

Core entities / classes

Class	Responsibility
Mark (enum)	X, O, EMPTY
Board	Grid state; place/validate mark; check win/draw
Player (ABC)	Name, mark, <code>choose_move(board) -> (row, col)</code>
HumanPlayer	Reads from provided input (injectable for testing)
RandomAIPlayer	Picks a random empty cell
WinChecker	Encapsulates win-detection logic (single responsibility)
Game	Orchestrates turn loop; holds players + board

Class diagram (ASCII UML)

DIAGRAM

```

+-----+
|   Game   |
+-----+
| board    |
| players  |
| checker  |
+-----+
| play()   |
+-----+
|
+-----+-----+
|   |   |   |   |   |
+---+---+---+---+---+
| Board | | WinChecker |
+-----+ +-----+
| grid  | | is_win()   |
| place() | | is_draw() |
+-----+ +-----+

<<ABC>>
+-----+
| Player |
+-----+
| mark   |
| choose_move()
+-----+
^
+-----+-----+
|   |   |   |   |   |
+-----+-----+
HumanPlayer RandomAIPlayer

```

Design patterns used

- **Strategy** — `Player.choose_move` makes each player type a strategy for move selection.
- **Template Method** — `Game.play()` defines the fixed turn loop; player-specific behavior is deferred to `choose_move`.
- **Single Responsibility** — `WinChecker` is separate from `Board` (board manages state; checker evaluates it).

- **Command** (mention) — For undo, wrap each move in a Command object with execute/undo.

Full code

```

from __future__ import annotations

import random
from abc import ABC, abstractmethod
from enum import Enum, auto
from typing import Optional

# ----- #
# Mark                                           #
# ----- #

class Mark(Enum):
    EMPTY = "."
    X = "X"
    O = "O"

# ----- #
# Board                                           #
# ----- #

class Board:
    def __init__(self, size: int = 3) → None:
        self.size = size
        self.grid: list[list[Mark]] = [
            [Mark.EMPTY] * size for _ in range(size)
        ]
        self._moves_made = 0

    def place(self, row: int, col: int, mark: Mark) → None:
        if not (0 ≤ row < self.size and 0 ≤ col < self.size):
            raise ValueError(f"Position ({row},{col}) out of bounds")
        if self.grid[row][col] ≠ Mark.EMPTY:
            raise ValueError(f"Position ({row},{col}) already occupied")
        self.grid[row][col] = mark
        self._moves_made += 1

    def is_full(self) → bool:
        return self._moves_made == self.size * self.size

    def empty_cells(self) → list[tuple[int, int]]:
        return [
            (r, c)
            for r in range(self.size)
            for c in range(self.size)
            if self.grid[r][c] == Mark.EMPTY
        ]

    def display(self) → None:
        header = " " + " ".join(str(c) for c in range(self.size))
        print(header)
        for r, row in enumerate(self.grid):
            print(f"{r} " + " ".join(cell.value for cell in row))
        print()

# ----- #
# Win Checker                                     #
# ----- #

class WinChecker:
    """Stateless win-detection. Separated from Board (SRP)."""

    def is_winner(self, board: Board, mark: Mark) → bool:
        size = board.size
        g = board.grid

        # Rows
        for row in g:
            if all(cell == mark for cell in row):
                return True

        # Columns
        for col in range(size):

```

```

        if all(g[row][col] == mark for row in range(size)):
            return True
        # Main diagonal
        if all(g[i][i] == mark for i in range(size)):
            return True
        # Anti-diagonal
        if all(g[i][size - 1 - i] == mark for i in range(size)):
            return True
        return False

def is_draw(self, board: Board) → bool:
    return board.is_full()

# ----- #
# Players                                     #
# ----- #

class Player(ABC):
    def __init__(self, name: str, mark: Mark) → None:
        self.name = name
        self.mark = mark

    @abstractmethod
    def choose_move(self, board: Board) → tuple[int, int]:
        ...

    def __repr__(self) → str:
        return f"Player({self.name},{self.mark.value})"

class HumanPlayer(Player):
    """In tests, inject a moves iterator so no stdin needed."""

    def __init__(
        self,
        name: str,
        mark: Mark,
        moves_iter: Optional[iter] = None, # type: ignore[type-arg]
    ) → None:
        super().__init__(name, mark)
        self._moves_iter = moves_iter

    def choose_move(self, board: Board) → tuple[int, int]:
        if self._moves_iter is not None:
            return next(self._moves_iter)
        while True:
            try:
                raw = input(f"{self.name} ({self.mark.value}), enter row col: ")
                row, col = map(int, raw.strip().split())
                return row, col
            except (ValueError, EOFError):
                print(" Invalid input. Enter two integers: row col")

class RandomAIPlayer(Player):
    def __init__(self, name: str, mark: Mark, seed: Optional[int] = None) → None:
        super().__init__(name, mark)
        self._rng = random.Random(seed)

    def choose_move(self, board: Board) → tuple[int, int]:
        empties = board.empty_cells()
        if not empties:
            raise RuntimeError("No moves available")
        choice = self._rng.choice(empties)
        print(f" {self.name} plays ({choice[0]},{choice[1]})")
        return choice

# ----- #
# Game                                       #
# ----- #

class GameResult(Enum):
    WIN = auto()
    DRAW = auto()
    IN_PROGRESS = auto()

```

```

class Game:
    """
    Orchestrates the turn loop.
    Decoupled from I/O via injectable players.
    """

    def __init__(
        self,
        player1: Player,
        player2: Player,
        board_size: int = 3,
    ) → None:
        if player1.mark == player2.mark:
            raise ValueError("Players must have different marks")
        self.players = [player1, player2]
        self.board = Board(size=board_size)
        self.checker = WinChecker()
        self._current_idx = 0
        self.winner: Optional[Player] = None

    @property
    def current_player(self) → Player:
        return self.players[self._current_idx]

    def play_turn(self) → GameResult:
        player = self.current_player
        row, col = player.choose_move(self.board)
        self.board.place(row, col, player.mark)
        self.board.display()

        if self.checker.is_winner(self.board, player.mark):
            self.winner = player
            print(f" {player.name} wins!")
            return GameResult.WIN

        if self.checker.is_draw(self.board):
            print(" It's a draw!")
            return GameResult.DRAW

        self._current_idx = 1 - self._current_idx
        return GameResult.IN_PROGRESS

    def play(self) → Optional[Player]:
        """Run the full game loop. Returns winner or None on draw."""
        self.board.display()
        while True:
            result = self.play_turn()
            if result != GameResult.IN_PROGRESS:
                break
        return self.winner

if __name__ == "__main__":
    # Scripted demo: two AI players with fixed seeds for reproducibility
    p1 = RandomAIPlayer("Alice", Mark.X, seed=42)
    p2 = RandomAIPlayer("Bob", Mark.O, seed=99)

    game = Game(p1, p2, board_size=3)
    winner = game.play()
    if winner:
        print(f"Winner: {winner.name}")
    else:
        print("Game ended in a draw")

    # Human vs AI example (uncomment for interactive play):
    # human = HumanPlayer("You", Mark.X)
    # ai = RandomAIPlayer("Bot", Mark.O)
    # Game(human, ai).play()

```

Extensibility discussion

- **N-in-a-row win (Connect-4 style):** Parameterize WinChecker with `win_length < size`. Only the checker changes.

- **More than 2 players:** Change `self.players` to a list; cycle with `(current_idx + 1) % len(players)`. Board already supports arbitrary marks.
- **AI with Minimax:** Replace `RandomAIPlayer.choose_move` with a minimax search tree. The `Player` interface is unchanged.
- **Undo last move:** Wrap `board.place` in a `MoveCommand(Command)`. `game.undo()` calls `command.undo()` which clears the cell and decrements `_moves_made`.
- **Networked game:** `RemotePlayer(Player)` overrides `choose_move` to send the board state over a socket and wait for a response.

Interview takeaway: The WinChecker separation is a key OOP point — "Board manages state; WinChecker evaluates it." This satisfies SRP and makes unit testing trivial. Always mention the injectable moves iterator for testing HumanPlayer without stdin.

Common Patterns Across LLD Problems

Entity-extraction checklist

When you get a new problem, run through this in 5 minutes:

1. **Nouns → candidate classes:** Underline every noun. Parking Lot → ParkingLot, Level, Spot, Vehicle, Ticket, Fee.
2. **Verbs → candidate methods:** Circle every verb. "find available spot" → `find_spot(vehicle)`.
3. **Prune attributes:** Nouns that are just data fields (name, price, timestamp) become fields, not classes.
4. **Find the variability:** What changes? Pricing? Split type? Scheduling algorithm? That variability → Strategy pattern.
5. **Find the lifecycle:** What has state? Elevator (Idle → Moving), Ticket (Open → Closed) → State pattern or enum.
6. **Find the singleton:** What is there exactly one of? ParkingLot, ElevatorSystem → Singleton.
7. **Find the family:** Vehicles, Players, Strategies → Factory + ABC.

Which pattern fits which need

Need	Pattern	Example in this file
Swap algorithm at runtime	Strategy	Pricing, Split, Scheduler, Rate Limiter, Player
Object with lifecycle phases	State	Elevator (IDLE/MOVING/MAINTENANCE)
One global instance	Singleton	ParkingLot, ElevatorSystem
Create family of objects	Factory Method	VehicleFactory
Fixed algorithm, variable steps	Template Method	Vehicle.allowed_spot_types, Game.play
Undo / replayable actions	Command	Board moves (TicTacToe extension)
Notify many on change	Observer	ElevatorSystem → FloorDisplay
Wrap object with extra behavior	Decorator	Rate limiter middleware layer

Revision cheat sheet

DIAGRAM

```

LRU Cache      → HashMap + DLinkedList + sentinel head/tail
Parking Lot    → Strategy(pricing) + Factory(vehicle) + Singleton(lot)
Rate Limiter    → Strategy(algorithm) + per-client state
Splitwise      → Strategy(split) + greedy debt simplification
Elevator       → State machine + Strategy(scheduler) + SCAN algorithm
Tic-Tac-Toe    → Strategy(player) + SRP (Board vs WinChecker)
  
```

The five questions interviewers ask at the end

1. "How would you make this thread-safe?" → Locks per shared resource; mention `threading.Lock / concurrent.futures`.
2. "How would you scale this to 1M users?" → Move to Redis/DB-backed storage; horizontal sharding; mention CAP theorem for rate limiter.
3. "Add feature X — what changes?" → Should be "add a new class, zero changes to existing" (OCP). If it isn't, your design has a seam problem.
4. "How would you test this?" → Unit test each strategy independently; mock dependencies; inject test doubles (e.g., moves iterator in TicTacToe).
5. "What did you deprioritize?" → Show you know the tradeoffs: "I skipped persistence, thread-safety on the outer loop, and error telemetry. For production I'd add X."

Final interview takeaway: In a 45-min round, a clean, running 200-line solution with 4 well-named classes beats a 500-line "enterprise" solution with 12 interfaces that doesn't run. Write the smallest correct design, name the patterns you used, and show one extensibility path. That's the formula.

ENGINEERING ESSENTIALS

Master the **fundamentals**.

Understand the **why** before the **how**. Connect every concept to a real system, an analogy, and a trade-off you can defend.

Vol. 17 — LLD Machine Coding

FAANG Interview Master Series